Section A: Loops (7 questions)

A1. What is the output?

```
None
i = 0
while i < 3
  print i
  i += 1
end
```

- a) `123`
- b) `012`
- c) `0123`
- d) Infinite loop

A2. Which loop runs **at least once**, regardless of the condition?

- a) `while cond ... end`
- b) `until cond ... end`
- c) `begin ... end while cond`
- d) `for i in ... end`

A3. What is the output?

```
None
3.times { |n| print n }
```

- a) `123`
- b) `012`
- c) `111`
- d) `333`

A4. What does `next` do inside a loop?

- a) Exits the loop entirely
- b) Skips to the next iteration
- c) Restarts the current iteration
- d) Restarts the entire loop from the beginning

A5. What is the output?

```
None
(1..5).each do |i|
  break if i == 3
  print i
end
```

- a) `12`
- b) `123`
- c) `1245`
- d) `45`

A6. Predict the output:

```
None
result = [1, 2, 3, 4].each { |x| x * 2 }
p result
```

- a) `[2, 4, 6, 8]`
- b) `[1, 2, 3, 4]`
- c) `nil`
- d) `8`

A7. True or False: `loop do ... end` will run forever unless you use `break` (or raise/return) inside it.

Section B: Flow Control (6 questions)

B1. Which values are **falsy** in Ruby?

- a) `false`, `nil`, `0`, `""`
- b) `false`, `nil`, `0`
- c) `false` and `nil` only
- d) `false`, `nil`, `""`, `[]`

B2. What is the output?

```
None
x = 0
puts "yes" if x
```

- a) Nothing
- b) `yes`
- c) `false`
- d) Error

B3. `unless` is equivalent to:

- a) `if !condition`
- b) `while !condition`
- c) `else`
- d) `elsif`

B4. What is the output?

```
None
grade = 85
result = case grade
         when 90..100 then "A"
         when 80..89  then "B"
         when 70..79  then "C"
         else "F"
         end
puts result
```

- a) `A`
- b) `B`
- c) `C`
- d) `F`

B5. Rewrite this using a ternary operator (write your answer):

```
None
if count > 10
  status = "high"
```

```
else
  status = "low"
end
```

B6. What does this print, and why?

```
None
puts "found" if nil == false
```

Section C: Variable Scope (6 questions)

C1. Match each prefix to its variable type:

Prefix	Type
`x`	?
`@x`	?
`@@x`	?
`$x`	?
`X`	?

C2. What happens?

```
None
x = 10
def show
  puts x
end
show
```

- a) Prints `10`
- b) Prints `nil`
- c) `NameError` / `NoMethodError` — undefined local variable

- d) Prints `0`

C3. What is the output?

```
None
class Counter
  @@count = 0
  def initialize
    @@count += 1
  end
  def self.count
    @@count
  end
end

Counter.new
Counter.new
puts Counter.count
```

- a) `0`
- b) `1`
- c) `2`
- d) Error

C4. What does an **uninitialized instance variable** (`@name` never assigned) return when read?

- a) Raises `NameError`
- b) `nil`
- c) `0`
- d) Empty string

C5. True or False: Blocks (`each do |x| ... end`) can access local variables defined outside the block, but method definitions (`def`) cannot.

C6. Why are class variables (`@@var`) generally discouraged in Rails codebases, and what is the common alternative?

Section D: Methods (6 questions)

D1. What is the difference between these two definitions?

```
None
class Report
  def generate; end        # (1)
  def self.generate; end   # (2)
end
```

Write how you'd call each.

D2. What is the output?

```
None
def greet(name = "Guest")
  "Hello, #{name}"
end
puts greet
```

- a) Error — missing argument
- b) `Hello,`
- c) `Hello, Guest`
- d) `Hello, nil`

D3. What does this method return?

```
None
def calc(a, b)
  sum = a + b
  sum * 2
  "done"
end
```

- a) `sum * 2`
- b) `"done"`
- c) `nil`
- d) `sum`

D4. What is the output?

```
None
def process(*args)
  args.length
end
puts process(1, 2, 3, 4)
```

- a) `1`
- b) `4`
- c) `[1, 2, 3, 4]`
- d) Error

D5. Inside an **instance method**, what does `self` refer to? Inside a **class method**?

D6. What is the output?

```
None
class User
  def name
    "chandra"
  end

  def display
    name.capitalize
  end
end

puts User.new.display
```

- a) `chandra`
- b) `Chandra`
- c) `NameError`
- d) `nil`